Lesson 10: Database Transactions

Database Transaction

- Any action that reads from or writes to a database
- A logical unit of work that must be completed or aborted in it's entirety, there is no middle ground
- Changes the state of the ddata in a database from one to another while satisfying all data integrity constraints
- Database transactions are composed of one or more requests; each request is represented by each operation

Database Transaction Properties and ACID

- Atomicity
- Consistency
- Isolation
- Durability

Atomicity

- All operations of a transaction must be completed or the transaction is aborted

Consistency

- Refers to the permanence of a database state and the fact that a transaction takes a database from one state to another
- Each state must not violate any data integrity constraint or it is aborted

Isolation

- The data utilized by one transaction cannot be used by another until the first transaction is completed
- Particularly important in the realworld where most databases are utilized by multiple users and multiple applications simultaneously

Durability

- When transactions are completed and commited they cannot be undone or lost

ACID Example

- Let's look at an example using our baseball data

Serializability

- While not a property included in ACID, serializability is critical!
- Since we expect concurrent transaction in our RDBMS, **serializability** ensures that the scheduling and ordering of concurrent transaction operations are consistent.

ANSI SQL Transactions

1. An explicit `COMMIT` statement is reached and all changes are recorded to the database.
2. An explicity `ROLLBACK` statement is reached, changes are not recorded and the database is returned to it's previous state.
3. An implicit `COMMIT`. The end of SQL script is reached without error and all changes are recorded in the database.
4. An implicit `ROLLBACK`. A SQL script terminates abnormally, aborting the transaction. Changes are not recorded and the database is returned to it's previous state.

Transaction Logs

- All RDBMs ensure durability by using logs to store all changes to the data in database
  - Think changes made with DML using `INSERT`, `UPDATE` and `DELETE`
- The tranaction log is managed separately from the database files themseleves
- Transaction logs track:
  - The type of operation: `INSERT`, `UPDATE` or `DELETE`
  - The objects (tables) affected by the transaction
  - The values of the fields before and after; the before state and after state
  - Pointers to the previous and next transaction log entries for the same transaction

Transaction Logs Continued

- When designing and implementing a database, how transactions logs are managed backed up is crtical
    - Transaction logs can "get big" and cause disk space to fill
- Depending on the DBMS, sometimes logs can be placed on a seperate disk
- Logs can be replicated at interval of time in part or full

Concurrency Control

- Unlike our lab enviornment, almost every real-world implementation of a database involves multiple users including actual people and applications
- This results in the need to coordinate the simultaneous execution of transactions in this environment
- The RDBMS needs to be able to ensure the serializability of transactions to preserve data integrity and consistency

Concurrency Problems

- Three main issues occur within concurrency control
- All occur when more than one transaction is updating the same row(s) and column(s) of data resulting

1. Lost Updates
2. Uncommitted Data
3. Inconsistent Retrievals

Lost Updates

- One of the updates is "lost" because it is overwritten by the other transaction
- Let's look at an example

Uncommitted Data

- Occurs when the first transaction is rolled back after the second transaction has already read (and used) uncommitted data.
- Violates the isolation property!
- Let's look at an example

Inconsistent Retrievals

- Occurs when a transaction accesses data before and after one or more transaction have finished working with the data.
- Violates the isolation property!
- Let's look at an example

Concurrency Control and the Scheduler

- DBMS have internal schedulers to to establish the order of operations of concurrent transactions.
    - As usual, their implementation differs between DBMS
- The purpose of the scheduler is to create a serializable schedule such that the interleaved execution of the concurrent operations yields the same result as if the transactions were executed in serial order.

Concurrency Control Methods

We'll cover three main methods that have been implemented for concurrency control

1. Locking Methods
2. Time Stamping
3. Optimistic Methods

Locking Methods

- A **lock** guarantees the exclusive use of data to the current transaction
- **Pessimistic locking** is a common approach that is based on the assumption that the conflict between transactions is likely rather than edge case
- Most DBMSs have locking built into the software in the form of a **lock manager** that is responsible for assigning and policing the locks used by transactions

Locking Methods, Granularity

Ordered Most Least to Most Granular

1. Database Level
2. Table Level
3. Page Level
4. Row Level
5. Field Level

Locking Methods, Granularity Continued

- Database Level Locks
  - The entire database is locked while a transaction takes place
  - Impractical for most scenarios
- Table Level Locks
  - A entire table is locked while a transaction takes place, preventing any other transaction from accessing or using the data in that table
  - More practical than database level locking, but can still create numerous delays

Locking Methods, Granularity Continued

- Page Level Locks
  - Database data is stored pages (aka diskpages), which are directly addressable sections of disk (physical storage) that store the data for one or more rows from one or more tables
  - Locks a page or multiple pages while a transaction takes place
  - **The most common type of locking implemented**
- Row Level Locks
  - A specific row of data is locked while a transaction takes place
  - More robust than page level locks
  - High overhead to implement in the DBMS

Locking Methods, Granularity Continued

- Field Level Locks
  - A specific row and field of data is locked while a transaction takes place
  - More robust than row level locks
  - Highest overhead to implement in the DBMS

Locking Methods, Lock Types

- Binary Lock
  - Locks have two values: Unlocked (0) and Locked (1)
  - The DBMS locks and unlocks the data
- Exclusive Lock
  - Exists when access to data is reserved for a specific transaction
  - These locks can be held by one and only one transaction
- Shared Lock
  - Exists when multiple transactions are granted **read only** access to data

Two Phase Locking (2PL)

Two-phase locking helps to guarantee serializability by defining how transactions acquire and relinquish locks. The two phases are:

1. A growing phase where the transaction gathers all of the locks needed without unlocking any data (known as the lock point)
2. A shrinking phase where the transaction releases all locks and cannot obtain more

Two Phase Locking (2PL) Continued

- 2PL is governed by the following rules
  - Two transactions cannot have conflicting locks
  - No unlock operation can preced a lock operation within the same transaction
  - No data is changed until all locks are obtained (the transaction reaches lock point)

Deadlocks

- Occur when two or more transactions wait indefinitely for one another to relinquish their locks
- Deadlocks can be controlled through:
    - Deadlock preventions - a transaction requesting a new lock is aborted and rescheduled when there is a possibility for a deadlock to occur
    - Deadlock detection - the DBMS periodically checks for deadlocks and aborts and reschedules one or more of the competing transaction (victim transactions)
    - Deadlock avoidance - all locks must be obtained by a transaction before execution

Time Stamping

- An approach that schedules transactions using a globally unique timestamp.
- Uniqueness ensures that there is no equal timestamps between transactions
- Montonicity ensures that the timestamps only increase between transactions
- This approach requires two additional time stamp fields one for the last time read and one for the last update

Time Stamping Continued

- Wait/Die
  - If two transactions are competing for a lock, the younger transaction will die and be rescheduled with the same timestamp
  - The older transaction will always obtain the lock
- Wound/Wait
  - If two transactions are competing for a lock, the older transaction will preempt (wound) the younger transaction die and be rescheduled with the same timestamp
  - Again the older transaction will always obtain the lock

Optimistic Methods

- Approaches based on the assumption that the conflict between transactions is an edge case rather than likely
- Transactions move through through two or more phases, generally read, validation and write
- In the read phase, a private copy of the data is made and the updates performed
- In the validation phase, the changes are checked to ensure they don't cause data integrity or consistency issues
- In the write phase, the changes are permanently applied to the database

ANSI Isolation Levels Types of Reads

- Isolation levels are based on the type of data transactions can see during execution
  - A **dirty read** occurs when a transaction can read uncommitted data
  - A **nonrepeatable read** occurs when the same row is read and different results are returned based on a deletion or update
  - A **phantom read** occurs when a transaction executes the same query at multiple times with subsequent queries yielding additional rows

ANSI Isolation Levels

The degree to which data of one transaction is isolated from other concurrent transactions

- Read Uncommitted (least restrictive)
  - Transactions can read committed and uncommitted data with no locking
  - Allows dirty reads, nonrepeatable reads and phantom reads
- Read Committed
  - Transactions can only read data that has been committed
  - This is is the default mode in which most DBMSs operate
  - Allows nonrepeatable reads and phantom reads

ANSI Isolation Levels Continued

- Repeatable Read
  - Ensure queries return consistent results using shared locks to ensure other transactions don't read update a row until after it is read by the initial transaction
  - Only allows phantom reads
- Serializable (most restrictive)
  - The most restrictive, preventing dirty reads, nonrepeatable reads and phantom reads

Database Recovery Management

- Database recovery restores a database from a previous consistent state
  - Necessary in the event of human error, natural disasters or hardware/software failure
- Important Concepts
  - The write-ahead-log (WAL) protocol ensures that transactions are written to the log before the database is updated
  - Redundant transaction logs ensure that multiple copies of the transaction log are stored and can be swapped
  - A database buffer is temporary in-memory storage of data being used in a transaction that is cleared once the transaction is committed
  - Checkpoints are operations in which the DBMS writes all data in buffers to memory, it happens regularly behind the scenes

Database Recovery Continued

- In a deferred-write recovery procedure, transaction operations do not immediately update the database, instead the transaction log is updated and then those events propogated to the database.
- If a failure occurs the recovery process follows these steps:

1. Identify the last checkpoint
2. Any transactions committed before the last checkpoint are safe
3. All commits made after the last checkpoint need to made in order from oldest to newest
4. Any transactions that were not committed or rolled back can be skipped because no updates were made

Database Recovery Continued

- In a deferred write-through, transactions immediately update the database even before the commit is made. A rollback is done if an transcation is subsequently aborted.
- If a failure occurs the recovery process follows these steps:

1. Identify the last checkpoint
2. Any transactions committed before the last checkpoint are safe
3. All commits made after the last checkpoint need to made in order (from oldest to newest)
4. Any transactions that were not committed or rolled back are executed in order from newest to oldest

Homework

- Read Chapter 11